

ITSOLERA CYBERSECURITY DEPARTMENT

Red Team — Offensive Security Internship 2026

CVE-2021-3156

Baron Samedit

Heap-Based Buffer Overflow in sudo — Local Privilege Escalation

Company	ITSOLERA PVT LTD
Project	CVE Exploit Research & Development
CVE	CVE-2021-3156 (Baron Samedit)
CVSS Score	7.8 — HIGH
Due Date	14th August 2026
Classification	CONFIDENTIAL — Educational Use Only

1. Executive Summary

CVE-2021-3156, nicknamed Baron Samedit by the Qualys research team who discovered it, is a critical heap-based buffer overflow vulnerability in sudo — one of the most fundamental security utilities in Linux. Assigned a CVSS v3 score of 7.8 (High), it allows any local user to escalate privileges to root (UID 0), regardless of whether they have any sudo permissions configured in /etc/sudoers.

The vulnerability resided undetected in sudo's source code for approximately 10 years, from the introduction of the vulnerable code in July 2011 (sudo 1.8.2) until its public disclosure on January 26, 2021. Its scope is enormous: virtually every major Linux distribution shipped with a vulnerable sudo version at the time of disclosure, affecting millions of servers, desktops, and cloud instances worldwide.

Property	Value
CVE ID	CVE-2021-3156
Nickname	Baron Samedit
CVSS v3 Score	7.8 — High
Discoverer	Qualys Research Team
Affected Versions	sudo 1.8.2 – 1.8.31p2, 1.9.0 – 1.9.5p1
Patched In	sudo 1.9.5p2, sudo 1.8.32
Auth Required	Local user only (no sudo access needed)
Type	Heap-based buffer overflow → Local Privilege Escalation
Disclosure	2021-01-26
CVE Present Since	2011-07-09 (sudo 1.8.2)

2. CVE Description & Background

2.1 What is sudo?

sudo (superuser do) is a standard Unix/Linux utility that allows a permitted user to execute commands as another user — typically root. It is one of the most essential security tools on Linux systems: administrators grant limited, audited elevated access to users without sharing the root password.

2.2 sudoedit and Shell Mode

sudoedit is a companion to sudo that allows users to edit files with elevated privileges. The -s (shell mode) flag applies escape processing to command arguments, making them safe to pass to a shell. This argument transformation — specifically the escape handling — contains the vulnerability.

2.3 Affected Software Versions

Branch	Vulnerable Range	Patched Version
sudo 1.8.x	1.8.2 through 1.8.31p2	1.8.32
sudo 1.9.x	1.9.0 through 1.9.5p1	1.9.5p2

3. Root Cause Analysis

3.1 The Vulnerable Function — set_cmd()

The vulnerability exists in the set_cmd() function in src/sudo.c. When sudo processes arguments in shell mode (-s flag), it applies escape handling to each argument. The function first calculates the required buffer size, then allocates a heap buffer (cmd_args), and finally copies the escaped data into it.

The bug is a mismatch between the size calculation and the actual copy:

```
// SIZE CALCULATION — correctly handles '\' when from[1] != '\0'
if (from[0] == '\\' && from[1] != '\0') { from++; } // skip backslash
cmd_size++; // counts the escaped character

// COPY STEP — DIFFERENT behaviour when from[1] == '\0'
if (from[0] == '\\' && from[1] != '\0') {
    from++; // ✗ when arg ends with '\', this reads PAST null
    terminator
    *out++ = *from; // writes the byte AFTER the argument string
}
```

3.2 The Off-By-One Explained

When an argument ends with a backslash (e.g. 'AAAA\'), the copy step reads one byte past the null terminator of that argument. This byte — whatever happens to exist in heap memory after the argument string — is written into cmd_args beyond its allocated boundary.

With a carefully crafted argument (specific length to align the overflow with an adjacent heap object), an attacker can overwrite one byte of the next heap chunk. That byte, combined with heap grooming to control what sits adjacent, enables controlled heap corruption and ultimately code execution as root.

3.3 The Patch — sudo 1.9.5p2

The fix added a pre-validation check that rejects any argument ending with a trailing backslash before the vulnerable processing code is reached:

```
// PATCHED — sudo 1.9.5p2
if (len > 0 && (*to)[len-1] == '\\') {
    sudo_warnx("invalid argument"); // reject and exit
    debug_return_int(NOT_FOUND_ERROR);
}
```

4. CVSS v3 Score Breakdown

Overall Score: 7.8 (High) | Vector: CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Metric	Value	Justification
Attack Vector	Local (AV:L)	Attacker must have local shell access (SSH, console)
Attack Complexity	Low (AC:L)	No race conditions, special timing, or knowledge required
Privileges Required	Low (PR:L)	Any local user — no sudo permissions needed at all
User Interaction	None (UI:N)	Fully automated — no victim action needed
Scope	Unchanged (S:U)	Stays within OS privilege model
Confidentiality	High (C:H)	Root: read all files, /etc/shadow, private keys
Integrity	High (I:H)	Root: modify any system file, install backdoors
Availability	High (A:H)	Root: halt, crash, or corrupt the system
OVERALL	7.8 — HIGH	

5. Exploitation Steps

 **WARNING:** All steps must be performed **ONLY** inside the isolated Docker lab container. Never against real systems.

5.1 Start the Lab

```
cd CVE-2021-3156-Project/lab/  
docker compose up -d --build  
docker exec -it baron_samedit_target bash
```

5.2 Verify Vulnerability (Inside Container as labuser)

```
whoami          # Expected: labuser  
sudo --version  # Expected: Sudo version 1.8.31  
sudo -l         # Expected: not allowed (no sudoers entry)  
sudoedit -s '\\ ' 2>&1 # Expected: NOT 'usage:' – confirms vulnerable
```

5.3 Run exploit.py Check

```
python3 exploit.py --check-only  
# All checks should PASS including vulnerability canary
```

5.4 Run Safe Mode (no exploitation)

```
python3 exploit.py --safe-mode
```

5.5 Run LPE Exploit

```
python3 exploit.py --cmd 'id'  
# Expected output: uid=0(root) gid=0(root) groups=0(root)  
python3 exploit.py --cmd 'cat /etc/shadow' --output ../proof/terminal_logs.txt
```

6. Code Explanation (exploit.py)

6.1 Script Architecture

Function	Purpose
check_platform()	Confirm we are on Linux (exploit is Linux-specific)
check_sudo_version()	Parse sudo --version and compare against vulnerable range 1.8.2-1.8.31p2 / 1.9.0-1.9.5p1
check_sudoedit_present()	Confirm sudoedit binary is accessible in PATH
check_current_user()	Confirm we are NOT already root; show current UID for proof
get_system_info()	Collect glibc version, architecture — needed for heap strategy selection
run_canary_test()	Run 'sudoedit -s \\\' and parse output to confirm vulnerability without exploitation
select_heap_strategy()	TODO stub: select grooming strategy based on glibc version
build_overflow_argument()	TODO stub: craft the exact argument string that triggers the overflow
trigger_overflow_and_escalate()	TODO stub: launch sudoedit with crafted args and env vars → root command

6.2 The Vulnerability Canary

The canary test (run_canary_test) is the most important detection mechanism. It runs the minimal trigger command and categorises the response:

- Patched: output contains 'usage:' or 'invalid argument' — sudo rejected the bad input
- Vulnerable: output is anything else, including 'not a regular file', a segfault (exit -11), or other unexpected behaviour
- This is safe to run — it does not attempt any heap manipulation or exploitation

7. Proof of Exploitation

 **MEMBER 4 ACTION REQUIRED:** Run the lab, execute all steps, insert screenshots from proof/screenshots/ and terminal output below.

7.1 Evidence Checklist

Evidence Item	Screenshot File	Status
sudo --version showing 1.8.31	01_sudo_version.png	[] Captured
whoami / id as labuser (no root)	02_whoami_before.png	[] Captured
sudo -l showing 'not allowed'	03_sudo_l_denied.png	[] Captured
Canary test output (not 'usage:')	04_canary.png	[] Captured
exploit.py --check-only (all PASS)	05_checks_pass.png	[] Captured
exploit.py --cmd id → uid=0(root)	06_root_id.png	[] Captured
Patched container canary → 'invalid argument'	07_patched_fail.png	[] Captured

7.2 Expected Output After Successful Exploitation

```
[+] Platform: Linux x86_64
[+] sudoedit found at: /usr/bin/sudoedit
[+] sudo 1.8.31 is in the 1.8.x VULNERABLE range
[+] Running as: labuser (UID=1000) – ideal for LPE demo
[+] Canary → 'not a regular file' – VULNERABLE
[*] Phase 1/3: Selecting heap strategy...
[*] Phase 2/3: Building overflow argument...
[*] Phase 3/3: Triggering overflow → executing as root...
Command Output (id):
uid=0(root) gid=0(root) groups=0(root)
```

8. Mitigation & Remediation

8.1 Primary Fix — Upgrade sudo

 **RECOMMENDED:** Upgrade sudo to 1.9.5p2+ or 1.8.32+. This is the only complete fix.

```
# Ubuntu / Debian
sudo apt-get update && sudo apt-get install --only-upgrade sudo
sudo --version # Verify: should show 1.9.7+ on Ubuntu 22.04
```

8.2 Verify Patch Applied

```
sudoedit -s '\\ ' 2>&1
# PATCHED output: sudoedit: invalid argument
# VULNERABLE output: sudoedit: /\: not a regular file
```

8.3 Temporary Mitigations

- Restrict sudoedit permissions: `chmod 0700 /usr/bin/sudoedit` (breaks legitimate use)
- Deploy AppArmor profile for sudoedit to prevent `-s` flag usage
- Monitor auditd for 'sudoedit -s' invocations by non-admin users

8.4 Hardening Summary

Measure	Priority
Upgrade sudo to 1.9.5p2+ / 1.8.32+	 Critical
Enable unattended security updates	 High
Principle of least privilege on local accounts	 High
Deploy auditd rules for sudoedit -s invocations	 Medium
Enable AppArmor / SELinux mandatory access control	 Medium

9. Vulnerability Timeline

Date	Event
2011-07-09	Vulnerable code introduced in sudo 1.8.2 (shell mode escaping added)
2021-01-13	Qualys researchers internally discover the vulnerability
2021-01-26	Coordinated disclosure: sudo 1.9.5p2 and 1.8.32 patches released
2021-01-26	All major Linux distributions release security updates simultaneously
2021-01-27	blasty publishes first public PoC (C) within 24 hours of disclosure
2021-01-28	worawit publishes multi-strategy Python PoC covering 3 glibc versions
2021-02+	Active exploitation in the wild on unpatched enterprise Linux systems

10. References

Source	URL
Qualys Advisory (discoverers)	https://blog.qualys.com/vulnerabilities-threat-research/2021/01/26/cve-2021-3156-heap-based-buffer-overflow-in-sudo-baron-samedit
MITRE CVE-2021-3156	https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-3156
NVD Entry	https://nvd.nist.gov/vuln/detail/CVE-2021-3156
sudo Security Advisory	https://www.sudo.ws/security/advisories/unescape_overflow/
blasty PoC	https://github.com/blasty/CVE-2021-3156
worawit PoC (3 strategies)	https://github.com/worawit/CVE-2021-3156
ExploitDB #49521	https://www.exploit-db.com/exploits/49521
GTFOBins — sudo	https://gtfobins.github.io/gtfobins/sudo/

 **DISCLAIMER:** Produced for educational purposes as part of the ITSOLERA Red Team Internship. All testing was performed exclusively against an isolated Docker lab environment. No live or production systems were targeted.